Tutorial Prático: Transformando nosso Estoque com Orientação a Objetos

Nesta atividade, vamos construir um sistema de estoque inteligente. Em vez de usar várias variáveis e listas separadas, vamos criar um "molde" (a Classe) que guardará os dados do produto e também saberá fazer os próprios cálculos.

Abra o seu editor de código e siga os passos abaixo!

Passo 1: Criando o Molde do Produto

A primeira coisa que precisamos fazer é criar a nossa classe e definir quais informações todo produto deve ter assim que "nasce". Faremos isso usando o método construtor `__init__` .

Digite o código abaixo:

Python

```
class Produto:
    # O método __init__ é chamado assim que criamos um novo produto
    def __init__(self, nome, estoque_inicial, vendas):
        self.nome = nome
        self.estoque_inicial = estoque_inicial
        self.vendas = vendas
```

O que está acontecendo? O `self` representa o próprio produto. Estamos dizendo: "o nome deste produto recebe o nome que foi digitado", e assim por diante.

Passo 2: Ensinando o Produto a Calcular o Estoque

Agora que o produto já guarda os seus dados, vamos ensiná-lo a usar esses dados. Vamos criar um método (uma função dentro da classe) para calcular o estoque final.

Adicione este código dentro da classe, logo abaixo do `__init__` :

Python

```
    # Um método simples para calcular o estoque final
    def calcular_estoque_final(self):
```

```
        return self.estoque_inicial - self.vendas
```

Perceba a diferença: Não precisamos passar parâmetros como `(inicial, vendas)`. O método usa o `self` para pegar os dados que ele mesmo já guardou no Passo 1!

Passo 3: Ensinando o Produto a Verificar sua Situação

O nosso produto também precisa saber dizer se o estoque dele está bom, baixo ou zerado. Vamos criar outro método para isso.

Adicione abaixo do cálculo de estoque:

Python

```
    # Um método simples para verificar a situação do estoque
    def verificar_situacao(self):
        # Chamamos o método acima usando o 'self' para pegar o valor atualizado
        estoque_final = self.calcular_estoque_final()

        if estoque_final >= 10:
            return 'Estoque suficiente'
        elif estoque_final >= 1:
            return 'Estoque baixo'
        else:
            return 'Produto esgotado'
```

Passo 4: Criando o Relatório do Produto

Para fechar a nossa classe, precisamos que o produto saiba imprimir os seus próprios dados de forma organizada na tela.

Adicione este último método na classe:

Python

```
    # Método para imprimir os dados deste produto específico
    def exibir_relatorio(self):
        print('\n RELATÓRIO DE ESTOQUE')
        print(f'Produto: {self.nome}')
        print(f'Estoque inicial: {self.estoque_inicial}')
        print(f'Unidades vendidas: {self.vendas}')
```

```
        print(f'Estoque final: {self.calcular_estoque_final()}')
        print(f'Situação: {self.verificar_situacao()}')
```

Passo 5: Construindo o Menu e as Entradas (Fluxo Principal)

Agora vamos sair da classe (remova o recuo/indentação) e construir o programa principal. Aqui usaremos os comandos `while` e `input` que vocês já conhecem.

Digite o código abaixo no final do seu arquivo:

Python

```
# ========================================
# FLUXO PRINCIPAL DO PROGRAMA (FORA DA CLASSE)
# ========================================

produtos = []

while True:
    print("\n--- CADASTRO DE PRODUTO ---")

    # Coletamos os dados de forma procedural, como já sabemos
    nome_digitado = input('Digite o nome do produto: ')
    inicial_digitado = int(input('Digite a quantidade inicial em estoque: '))
    vendas_digitadas = int(input('Digite quantas unidades foram vendidas: '))
```

Passo 6: A Mágica da Orientação a Objetos (Instanciação)

Ainda dentro do laço `while`, vamos pegar os dados que o usuário acabou de digitar e criar o nosso objeto inteligente.

Python

```
    # Criamos um objeto "Produto" entregando as variáveis digitadas
    novo_produto = Produto(nome_digitado, inicial_digitado, vendas_digitadas)

    # Guardamos o objeto inteiro na lista
    produtos.append(novo_produto)

    continuar = input("\nDeseja cadastrar outro produto? (s/n): ").strip().lower()
```

```
    if continuar != "s":
        break
```

O que está acontecendo? Ao invés de criar dicionários complicados, a linha `Produto(...)` aciona aquele `__init__` que fizemos no Passo 1, empacotando tudo em uma variável só (`novo_produto`).

Passo 7: Exibindo o Relatório Geral

Para finalizar, fora do laço `while` , vamos percorrer a nossa lista de produtos e pedir para cada um deles executar o método de imprimir na tela.

Python

```
print("\n============= RELATÓRIO GERAL =============")

# Percorremos a lista de objetos e pedimos para cada um exibir seu próprio relatório
for p in produtos:
    p.exibir_relatorio()
```